

```
#!/usr/bin/env python3
"""Command-line interface for the phishing
detector.

Examples:
    python cli.py --file
examples/sample_phishing_email.txt
    python cli.py --text "Your account has been
suspended, click here: http://paypa1-secure.xyz"
    python cli.py --file
examples/sample_phishing_email.txt --no-api  #
heuristics only
"""

import argparse
import sys

from phishing_detector import PhishingAnalyzer,
RiskLevel

COLORS = {
    RiskLevel.LOW: "\033[92m",    # green
    RiskLevel.MEDIUM: "\033[93m", # yellow
    RiskLevel.HIGH: "\033[91m",   # red
}
RESET = "\033[0m"

def build_parser():
    parser =
    argparse.ArgumentParser(description="Phishing
Email & Message Detector")
    source =
    parser.add_mutually_exclusive_group(required=True
```

```

e)
    source.add_argument("--file", help="Path to
a .eml file or plain text message")
    source.add_argument("--text", help="Raw
message text to analyze")
    parser.add_argument("--no-api",
action="store_true",
                        help="Skip
PhishTank/VirusTotal lookups; heuristics only")
    parser.add_argument("--max-urls", type=int,
default=5,
                        help="Max number of
URLs to check against APIs (default: 5)")
    return parser

def print_report(result):
    risk = result.risk
    color = COLORS.get(risk.level, "")
    print("\n" + "=" * 60)
    print(f"RISK LEVEL: {color}
{risk.level.value.upper()}{RESET} (score:
{risk.score})")
    print("=" * 60)

    if result.parsed.is_structured_email:
        print(f"From:
{result.parsed.from_header or '(none)'}")
        print(f"Subject: {result.parsed.subject
or '(none)'}")

    print(f"URLs found:
{len(result.parsed.urls)}")

    if risk.reasons:
        print("\nSignals:")
        for reason in risk.reasons:

```

```

        print(f" - {reason}")
    else:
        print("\nNo phishing signals detected.")

    if result.phishtank_results or
result.virustotal_results:
        print("\nThreat intel:")
        for url in result.parsed.urls[:5]:
            pt =
result.phishtank_results.get(url)
            vt =
result.virustotal_results.get(url)
            print(f" {url}")
            if pt:
                if pt.get("error"):
                    print(f"    PhishTank:
error - {pt['error']}")
                else:
                    print(f"    PhishTank:
in_database={pt['in_database']} verified=
{pt['verified']}")
            if vt:
                if vt.get("error"):
                    print(f"    VirusTotal:
error - {vt['error']}")
                else:
                    print(f"    VirusTotal:
malicious={vt['malicious']} suspicious=
{vt['suspicious']}")
            print()

def main():
    args = build_parser().parse_args()

    if args.file:
        with open(args.file, "r", encoding="utf-

```

```
8", errors="replace") as fh:
    raw_text = fh.read()
else:
    raw_text = args.text

    analyzer =
PhishingAnalyzer(use_phishtank=not args.no_api,
use_virustotal=not args.no_api)
    result = analyzer.analyze(raw_text,
max_urls_checked=args.max_urls)
    print_report(result)

    exit_codes = {RiskLevel.LOW: 0,
RiskLevel.MEDIUM: 1, RiskLevel.HIGH: 2}
    sys.exit(exit_codes[result.risk.level])

if __name__ == "__main__":
    main()
```